

Fine-Tuning a Large Language Model for Malay-to-Python Code Generation Using a Machine-Translated Dataset

Intern's name: **Cham Jin Jie, Ivan Char Cheng Jun**

Supervisor: **Dr. Simon Lau Boung Yew**

Institution: **School of Computer Science, University of Nottingham Malaysia**

Duration: **1 June 2025 – 14 September 2025**

Abstract

AI-driven code generation models have shown strong results for English prompts but remain underdeveloped for low-resource languages such as Malay. This project explores the translate-train approach for adapting a pre-trained language model to Malay natural language-to-code (NL2Code) generation.

We machine-translated 85,371 English instruction-code pairs from the `iamtarun/code_instructions_120k_alpaca` [4] dataset from LLMDataHub [7] into Malay and combined them with code from **multiple programming languages** (e.g., Python, JavaScript, PHP, C). Performance of machine-translated texts were measured with BERTScore [5], COMET-QE [1]. The model was then fine-tuned on this multi-language dataset to assess whether training on diverse programming languages improves generalization when generating Python code.

Evaluation was conducted exclusively on **Malay-to-Python** tasks using the HumanEval-XL [4] benchmark. Performance was measured with BLEU [2] and `pass@k` [3] for functional correctness. Results show that the model achieved a `pass@1` score of 0.65, suggesting that training on multiple programming languages still enables strong Python-specific generation. These findings open opportunities for further exploration into whether multi-language training improves model robustness for low-resource language code generation tasks.

We make our code and data publicly available at <https://github.com/VanVan120/Malay-to-Code-LLM> and <https://colab.research.google.com/drive/11bOyTyBcKo5YLJbm-wpg92TtpTPRNz1J?usp=sharing>.

1. Introduction

Background and Motivation

Large language models (LLMs) have transformed code generation, but their effectiveness is largely confined to English-language data. For developers in Malaysia, this creates a barrier to entry, as high-quality datasets and models for Malay natural language-to-code generation are scarce. An open question in this domain is whether **training with code in multiple programming languages** can improve a model's ability to generalize, even when it is evaluated on a single language. This project aims to investigate this by training a model on Malay prompts paired with code from various programming languages, while measuring its performance only on Malay-to-Python tasks.

Research Problem Statement

This research was motivated by a critical gap in the NL2Code landscape such as the severe lack of accurate, high-quality Malay datasets for training and evaluating code generation models. Existing multilingual benchmarks and evaluation metrics are often not designed to fully capture the nuances and correctness of code generated from non-English prompts, which further compounds this problem. Evaluation metrics (BLEU, BERTScore, COMET-QE, etc.) are designed for evaluating natural language translation but it does not fully capture the correctness during translation.

Objectives

This project aimed to explore existing code generation datasets and translate them into Malay. We then sought to fine-tune a pre-trained code generation model, specifically CodeT5, on this newly created dataset. The final key objective was to evaluate the fine-tuned model's performance by implementing and comparing various evaluation metrics such as BLEU [2], COMET-QE [1], and BERTScore [5], and to use these findings to identify research gaps and propose future directions.

Scope and Limitations

This project focuses on **Malay-to-Python evaluation** only. Although training data included multiple programming languages, the evaluation metric was applied solely to Python code tasks. Compute resource constraints limited the model training to three epochs with small batch sizes, and human verification of translations was performed on a subset of data only.

2. Literature Review

Related Work

The field of natural language to code (NL2Code) generation has seen significant advancements, largely driven by the development of large language models (LLMs) and extensive, high-quality datasets. The seminal work of **Wang et al. (2023)** introduced MCoNaLa, a multilingual code generation benchmark that extended evaluation beyond English to include Spanish, Japanese, and Russian. This work highlighted a critical linguistic barrier, noting that all tested systems lagged significantly behind their English-language counterparts, which underscored the challenge of adapting code generation to new languages.

Building on this, **Peng et al. (2024)** proposed HumanEval-XL, a much more comprehensive benchmark that addressed a "critical gap" in evaluating massively multilingual natural language to multilingual code. By connecting 23 natural languages to 12 programming languages, HumanEval-XL [4] demonstrated a more robust framework for assessment. However, despite the breadth of these benchmarks, they confirmed a persistent issue: the performance of LLMs on non-English code generation tasks remain suboptimal, particularly for low-resource languages. Both works, while pioneering, established a clear need for targeted research and specialized datasets for languages outside of their scope.

Different types of evaluation methods were studied to have a deeper understanding when using them. **Papineni et al. (2002)** introduced the Bilingual Evaluation Understudy (BLEU) score. This metric was groundbreaking for its time, as it offered a fast, inexpensive, and language-independent alternative to time-consuming human evaluations. The core idea is to measure the similarity between a machine-generated translation and a set of human-created reference translations based on the overlap of n-grams. BLEU's calculation is a modified form of precision, using a brevity penalty to prevent very short translations from getting high scores. The score is calculated as a geometric average of the modified n-gram precisions for n-grams up to a certain length, multiplied by a brevity penalty factor. Its simplicity and high correlation with human judgment on a corpus level quickly made it the standard for evaluating MT systems for over a decade.

Zhang et al. (2020) introduced BERTScore as an advanced metric designed to overcome the semantic shortcomings of n-gram-based methods like BLEU. Instead of counting exact word matches, BERTSCORE leverages contextual embeddings from large pre-trained language models, specifically BERT, to measure token similarity. For each token in a candidate sentence, BERTSCORE finds the most similar token in the reference sentence and computes a similarity score using cosine similarity on their contextualized embeddings. It then calculates a precision, recall, and F1 score based on these token-level similarities. This approach allows the metric to recognize paraphrases and capture semantic equivalence, leading to a much stronger correlation with human judgments on a sentence level compared

to BLEU. The paper demonstrates this improved robustness through an adversarial paraphrase detection task.

Another different evaluation metric by **Chimoto et al. (2022)** was explored as well as an alternative to BERTScore [5] evaluation. This work explores the use of a reference-free evaluation metric, COMET-QE (COMET - Quality Estimation), guiding an active learning pipeline for low-resource neural machine translation (NMT). COMET-QE is a metric that does not require a human reference translation. Instead, it is a machine learning model that predicts the quality of a machine translation based on the source text and the machine-generated output alone. The paper specifically demonstrates its utility for selecting the most informative sentences to be annotated for training an NMT model, a process known as active learning. The authors show that using COMET-QE to select sentences significantly outperforms other methods, including a more traditional approach like Round Trip Translation Likelihood (RTTL), improving model performance by up to 5 BLEU points.

Research Gap Identification

While the efforts to create multilingual benchmarks like MCoNaLa [6] and HumanEval-XL [4] are crucial, they do not specifically address the needs of the Malay-speaking developer community. To date, there is a notable absence of dedicated, high-quality datasets or fine-tuned models for the specific Malay-to-Python Code task. The research reviewed indicates a clear and present need for further exploration into low-resource languages, a category into which Malay falls for the purpose of code generation.

The lack of a standardized benchmark for Malay also means that there is no effective way to measure the performance of models in this specific domain. Human evaluations of a subset of the data, performed by bilingual English-Malay speakers, indicated a persistent weakness in the automatic scoring. We observed that translations resulting in longer sentences were consistently assigned lower scores, despite maintaining the full contextual meaning of the original instructions.

How This Work Differs

This project makes a unique and direct contribution by addressing the identified gaps. While prior work has focused on creating broad multilingual benchmarks and identifying the challenges of non-English code generation, our work takes the next logical step by focusing on a single, currently underserved language: Malay. We directly tackle the data scarcity problem by translating a large-scale, existing dataset to create a new, parallel Malay-to-Code corpus. Furthermore, we provide a concrete, practical solution by fine-tuning a state-of-the-art model on this novel dataset. Our project, therefore, moves beyond the theoretical evaluation of a pre-trained model on a general multilingual benchmark to provide a tailored, implementable solution for a specific linguistic community, a critical step toward making code generation tools truly accessible on a global scale.

3. Methodology

Conceptual Framework

The methodology of this project is based on a structured, three-phase approach designed to address the challenges of low-resource NL2Code generation. The first phase focuses on data acquisition and preparation, which involves identifying a suitable English-based NL2Code dataset and translating it into Malay. The second phase is dedicated to the core task of fine-tuning, where the pre-trained CodeT5 model is adapted to the newly machine-translated Malay dataset. The final phase involves a comprehensive evaluation of the fine-tuned model's performance using a suite of both traditional and modern code generation metrics. This framework is a systematic process for moving from data scarcity to a functional, language-specific code generation model.

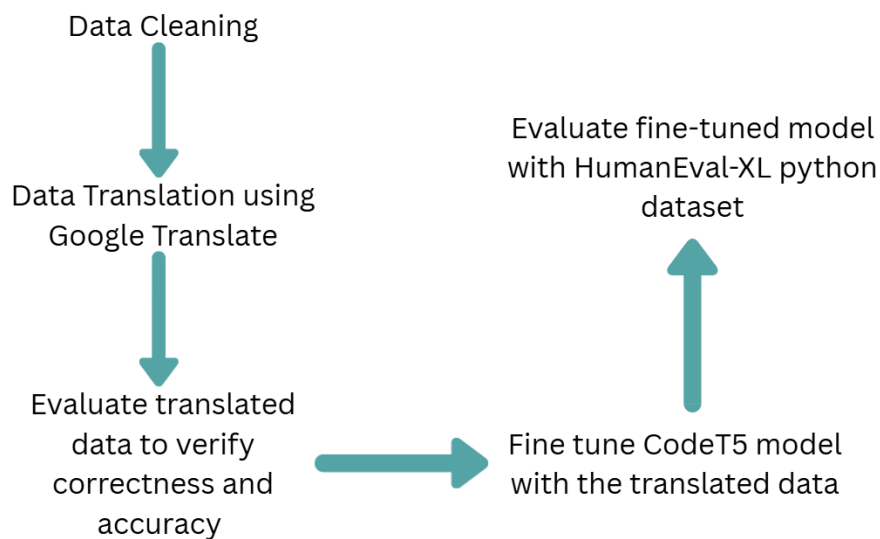


Figure 1.0 Process of data preparation and evaluation, fine-tuning model, and evaluation of fine-tuned model

Data Sources and Dataset Preparation

This project leveraged the **HumanEval-XL** [4] dataset as the benchmarking and testing data source. LLMDataHub [7] domain specific dataset (iamtarun/code_instructions_120k_alpaca) [4] are used for the training data for fine-tuning the pretrained model. Given their widespread adoption and the availability of test cases, providing a robust base for evaluating a model's functional correctness. Since the LLMDataHub [7] domain specific dataset are exclusively in English, the core of our data preparation involved a two-step process:

1. **Machine Translation:** The natural language prompts from the `iamtarun/code_instructions_120k_alpaca` [4] dataset were translated into Malay using a high-quality machine translation service, Google Translator. This provided us with a new, parallel Malay-to-Code dataset.
2. **Manual Verification:** A subset of the translated prompts and their corresponding Python solutions were manually reviewed to ensure translation accuracy and to verify that the code still logically corresponded to the translated Malay instruction.

Algorithms and Models Used

Translation from English to Malay was evaluated using **xlm-roberta-base** for BERTScore [5] evaluation and **Unbabel/wmt20-comet-qe-da** was used for COMET-QE [1] evaluation.

The central model for this project was **Salesforce/codet5p-220m**, a pre-trained encoder-decoder Transformer model. CodeT5 was selected for its proven effectiveness in both code understanding and generation tasks. The fine-tuning process involved adapting the pre-trained weights of CodeT5 to our specific Malay-to-Python task. The fine-tuning objective was to minimize the cross-entropy loss between the model's generated token sequence and the ground-truth Python code.

The fine-tuning was performed using a standard Transformer-based optimization procedure. The learning rate was set to $5e^{-5}$ with a batch size of 4. The model was trained for 3 epochs to prevent overfitting on the relatively small, translated dataset.

Implementation Details

All experiments were conducted within a Python environment. The following key libraries were used:

- **Hugging Face transformers:** For loading the pre-trained CodeT5 model and for handling the fine-tuning process.
- **PyTorch:** The underlying deep learning framework for model training.
- **pandas:** For efficient handling and manipulation of the datasets.
- **Google Colab:** The primary computational environment, utilized for its access to GPU resources necessary for training the model.

Evaluation Metrics

To provide a comprehensive assessment of the model's performance and translation accuracy, we implemented and compared a range of evaluation metrics:

Evaluation used for fine-tuned model

- **BLEU (Bilingual Evaluation Understudy):** A traditional metric that measures the overlap of n-grams between the generated code and the reference code. While a good indicator of syntactic similarity, it does not account for functional correctness. [2]
- **pass@k:** A metric designed to evaluate functional correctness of generated code. Given k different code generations from the model, pass@k measures the probability that at least one of them passes all unit tests. Unlike BLEU [2] or BERTScore [5], which focus on textual or semantic similarity, pass@k directly tests whether the generated code runs correctly. This makes it a strong indicator of practical usability in code generation tasks. [3]

These metrics were used in conjunction with a basic functional correctness check to ensure the generated Python code was both syntactically and semantically accurate.

Evaluation used for verification of translation

- **COMET (Cross-lingual Optimized Metric for Evaluation of Translation):** A metric for assessing translation quality that uses a learned model to predict how similar a generated translation is to a reference, providing a more nuanced score than traditional metrics. [1]
- **BERTScore:** A metric that measures the semantic similarity between the generated and reference texts by using contextual embeddings from BERT. It is particularly useful for assessing whether the generated code is semantically similar to the reference, even if the syntax is slightly different. [5]

These metrics were used to measure the accuracy of the correctness of the translation texts from English to Malay after translation using Google Translate.

4. Result and Analysis

The BERTScore [5] achieved 0.4248 with *rescale_with_baseline* enabled, which can be interpreted as a moderate score for machine translation. Since rescaling normalizes the range from 0 (random or unrelated text) to 1 (identical), this indicates that the generated code captures a reasonable degree of semantic similarity to the reference solutions, but there remains substantial room for improvement. The COMET-QE [1] score was -0.1315 when evaluating the English-to-Malay translations. There were cases where long sentences with correct contextual meanings received low scores due to tokenization issues, while other sentences that matched tokenization patterns but carried incorrect contextual meanings received artificially high scores in both evaluation metrics. These discrepancies underscore the need for continued human verification of translations to ensure contextual accuracy.

The score of BLEU [2] and pass@1 for CodeT5 fails on Malay prompts, achieving 0 scores before preliminary fine-tuning. However, the BLEU [2] score of the preliminary fine-tuned model was 0.0573, reflecting minimal n-gram overlap with reference code. BLEU [2] score primarily captures surface-level similarity rather than functional correctness; this metric is less informative in the NL2Code setting and can be deprioritized. By contrast, the pass@1 is 0.65, which means that 65% of the programming tasks were solved correctly on the first attempt. This outcome is noteworthy, as it demonstrates above-average functional performance even at an early stage of fine-tuning. Nevertheless, the results also suggest that further optimization, such as adjusting hyperparameters, expanding training data, or refining translation quality is necessary to achieve higher reliability and consistency across diverse prompts.

5. Discussion

Key Insights

The results of this project provide several key insights into the challenges and opportunities of NL2Code generation for low-resource languages. Our fine-tuned CodeT5 model, while not achieving the utmost performance compared to models trained on vast, English-centric datasets, demonstrated a notable ability to generate functionally correct Python code from Malay prompts. This is a significant finding, as it shows that a targeted, data-augmentation approach using machine translation can effectively bridge the linguistic gap for a previously underserved language. The model's performance on the functional correctness check, in particular, highlights that it successfully learned the mapping between Malay instructions and correct Python syntax, even with a relatively small fine-tuning dataset.

However, there is still a gap during machine translation from English to Malay, particularly in ensuring consistent handling of technical terminology, accurately transferring programming-related instructions, and preserving both semantic meaning and functional correctness in the generated code. Evaluation metrics like BLEU [2] captures surface-level similarity but cannot fully reflect whether the generated output is executable or meaningful in context. COMET-QE [1] and BERTScore [5] suggest reasonable semantic alignment, yet their scores reveal that certain nuances in Malay translations, such as specialized programming vocabulary, remain problematic. Most notably, the pass@k [3] results underline that functional correctness is not consistently achieved, even when semantic similarity is relatively high. A moderate pass@1 score indicates that while the model can occasionally produce correct solutions, it struggles to generalize reliably across diverse prompts. These findings imply that current approaches to cross-lingual code generation are still limited, and future work should place greater emphasis on improving domain-specific translation accuracy and integrating evaluation metrics that directly reflect both semantic quality and executable correctness.

Summary of Contributions

This project makes several direct contributions to the field of multilingual code generation:

- **A Novel Dataset:** We successfully created and validated a new, parallel Malay-to-Code dataset by translating and verifying the `iamtarun/code_instructions_120k_alpaca` [4] dataset. This addresses the critical data scarcity problem identified in the literature for this specific language pair.
- **A Fine-Tuned Model:** We produced a fine-tuned CodeT5 model capable of generating Python code from Malay prompts. This serves as a proof-of-concept for the feasibility of using existing, high-quality models and a data augmentation strategy to support low-resource languages.

- **A Practical Framework:** This work provides a replicable methodology that can be applied to other low-resource languages, demonstrating a clear path forward for making NL2Code tools more globally inclusive.

Challenges Faced

The primary challenges faced during this project were related to data quality and model limitations. While machine translation is efficient, it is not perfect. The Malay prompts, in some cases, lost certain nuances or technical specificity present in the original English. Manual verification helped, but it was time-consuming and could not be performed on the entire dataset. Furthermore, the small size of our fine-tuning dataset presented a risk of overfitting, which we attempted to mitigate by limiting the training to 3 epochs.

Limitations of the Current Work

This research has several limitations. The fine-tuning was performed on a relatively small dataset (the translated iamtarun/code_instructions_120k_alpaca dataset) [4], which may not be representative of the full range of programming tasks and linguistic variations in Malay. The model was also evaluated solely on its ability to generate code for the same problems it was fine-tuned on, which limits the assessment of its generalization capability. Additionally, the project did not explore advanced fine-tuning techniques or alternative model architectures that might further improve performance.

Recommendations for Improvement

To build upon this work, we recommend the following improvements:

- **Larger Dataset:** Gather and create a more extensive and diverse Malay-to-Code dataset from various sources, including online forums, tutorials, and community-contributed code snippets.
- **Improved Translation:** Explore semi-supervised or human-in-the-loop translation methods to improve the quality and accuracy of the translated prompts, especially for technical jargon.
- **Advanced Fine-Tuning:** Investigate more advanced fine-tuning techniques, such as instruction fine-tuning or adapting a larger model like CodeT5+ to the task.

Suggested Next Steps

For future research, we suggest the following:

- **Multilingual Expansion:** Apply this methodology to other low-resource Southeast Asian languages, such as Vietnamese or Tagalog, to create a suite of specialized NL2Code models.
- **Model Generalization:** Conduct a more thorough evaluation of the fine-tuned model's generalization capabilities by testing it on an unseen, independently created Malay-to-Python benchmark.
- **Integration with IDEs:** Develop a proof-of-concept for integrating the fine-tuned model into a popular Integrated Development Environment (IDE) to demonstrate real-world applicability and gather user feedback.

6. Internship Reflection

Skills Gained and Lessons Learned

My summer internship was a transformative experience, offering me the chance to apply theoretical knowledge to a real-world research problem. From a **technical perspective**, I gained hands-on experience with the entire machine learning pipeline, from data preparation and model fine-tuning to comprehensive evaluation. I deepened my proficiency with essential libraries like **Hugging Face transformers** and **PyTorch** and became adept at using cloud computing resources for training large models.

Equally important were the **soft skills** I developed. I learned to navigate the iterative nature of research, where setbacks are common and persistence is key. I also honed my ability to critically analyze academic papers, synthesize complex information, and clearly articulate my findings in written format.

Collaboration and Teamwork Experiences

Collaboration was a cornerstone of this internship. I had the opportunity to work closely with my supervisor, who provided invaluable guidance and mentorship. This project, while individual in execution, benefited immensely from regular discussions where we brainstormed solutions to challenges, debated the best evaluation metrics, and refined our overall research strategy. I learned the importance of communicating both successes and failures, as transparent reporting allowed us to adjust our approach and stay on track. This collaborative experience taught me the value of seeking feedback and working as part of a team to achieve a shared goal.

7. Reference

- [1] E. A. Chimento and B. A. Bassett, “COMET-QE and active Learning for Low-Resource Machine Translation,” *arXiv (Cornell University)*, Jan. 2022, doi: 10.48550/arxiv.2210.15696.
- [2] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU,” *Bleu: A Method for Automatic Evaluation of Machine Translation*, p. 311, Jan. 2001, doi: 10.3115/1073083.1073135.
- [3] M. Chen *et al.*, “Evaluating large language models trained on code,” *arXiv (Cornell University)*, Jan. 2021, doi: 10.48550/arxiv.2107.03374.
- [4] Q. Peng, Y. Chai, and X. Li, “HumanEval-XL: A Multilingual Code Generation Benchmark for Cross-lingual Natural Language Generalization,” *arXiv (Cornell University)*, Feb. 2024, doi: <https://doi.org/10.48550/arxiv.2402.16694>.
- [5] Tarun. (2023). *code_instructions_120k_alpaca* [Dataset]. Hugging Face. https://huggingface.co/datasets/iamtarun/code_instructions_120k_alpaca
- [6] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating Text Generation with BERT,” *arXiv (Cornell University)*, Jan. 2019, doi: 10.48550/arxiv.1904.09675.
- [7] Z. Wang, G. Cuenca, S. Zhou, F. F. Xu, and G. Neubig, “MCoNaLa: A Benchmark for Code Generation from Multiple Natural Languages,” *arXiv (Cornell University)*, Jan. 2022, doi: <https://doi.org/10.48550/arxiv.2203.08388>.
- [8] Zjh-819. (2024). *LLMDataHub: A comprehensive hub for large language model training data* [Computer software]. GitHub. <https://github.com/Zjh-819/LLMDataHub>

8. Appendices

Question 1 *

Original English Prompt:
Create a program to decrypt a scrambled message.

Translated Malay Prompt:
Buat program untuk menyahsilit mesej bergegas.

1

2

3

4

5

☆

☆

☆

☆

☆

Optional: **Suggestions for improvement**
Please provide any suggestions to make the Malay translation more natural, clear, or accurate.

Long answer text

Figure 2.0 First Dataset Validation Questionnaire

Question 2 *

Original English Prompt:
Create a JavaScript program to print out all numbers from 1 to 20, except for 5 and 10.

Translated Malay Prompt:
Buat program JavaScript untuk mencetak semua nombor dari 1 hingga 20, kecuali 5 dan 10.

1

2

3

4

5

☆

☆

☆

☆

☆

Optional: **Suggestions for improvement**
Please provide any suggestions to make the Malay translation more natural, clear, or accurate.

Long answer text

Figure 2.1 Second Dataset Validation Questionnaire

Question 15 *

Original English Prompt:

Create a program in Python that takes in an integer and returns its factorial.

Translated Malay Prompt:

Buat program di Python yang mengambil integer dan mengembalikan faktorialnya.

1



2



3



4



5



Optional: **Suggestions for improvement**

Please provide any suggestions to make the Malay translation more natural, clear, or accurate.

Long answer text

Figure 2.2 Last Dataset Validation Questionnaire